

Algorithmique & Programmation

Introduction à la programmation pour le TAL

Eric Jordan

14 novembre 2025

- 1 Les Fonctions
- 2 Faire ses propres fonctions
- 3 Exercices

1 Les Fonctions

2 Faire ses propres fonctions

3 Exercices

C'est quoi une fonction ?

- ▶ Du code déjà écrit, réutilisable

C'est quoi une fonction ?

- ▶ Du code déjà écrit, réutilisable
- ▶ Elles permettent surtout de décomposer nos programmes en plusieurs sous programmes

C'est quoi une fonction ?

- ▶ Du code déjà écrit, réutilisable
- ▶ Elles permettent surtout de décomposer nos programmes en plusieurs sous programmes
- ▶ On appelle une fonction avec des parenthèses et 0, 1 ou plusieurs *arguments*

C'est quoi une fonction ?

- ▶ Du code déjà écrit, réutilisable
- ▶ Elles permettent surtout de décomposer nos programmes en plusieurs sous programmes
- ▶ On appelle une fonction avec des parenthèses et 0, 1 ou plusieurs *arguments*
- ▶ La fonction nous renvoie **toujours** une valeur

C'est quoi une fonction ?

- ▶ Du code déjà écrit, réutilisable
- ▶ Elles permettent surtout de décomposer nos programmes en plusieurs sous programmes
- ▶ On appelle une fonction avec des parenthèses et 0, 1 ou plusieurs *arguments*
- ▶ La fonction nous renvoie **toujours** une valeur

Pour appeler à une fonction

```
nom_de_fonction(arguments_de_fonction)
```


Les fonctions natives de Python

On les a déjà vu :

- ▶ `print()`
- ▶ `input()`
- ▶ `type()`
- ▶ `len()`

Exemples

```
print("Bonjour")           # Bonjour
type(3.14)                  # <class 'float'>
len("Python")               # 6
```

Fonctions de conversion

- ▶ `str()`
- ▶ `float()`
- ▶ `list()`
- ▶ `set()`

Exemples

```
str(42)           # "42"
float("3.14")     # 3.14
list("abc")       # ['a', 'b', 'c']
set([1,2,2,3])    # {1, 2, 3}
```

Fonctions mathématiques

- ▶ `range()`
- ▶ `round()`

Exemples

```
list(range(5))           # [0, 1, 2, 3, 4]
round(3.14159, 2)        # 3.14
```

Fonctions pour les itérables

- ▶ `max()`
- ▶ `min()`
- ▶ `sum()`
- ▶ `enumerate()`

Exemples

```
max([1,5,3])           # 5
min([1,5,3])           # 1
sum([1,2,3])           # 6
list(enumerate(['a','b'])) # [(0, 'a'), (1, 'b')]
```

Sommaire

1 Les Fonctions

2 Faire ses propres fonctions

3 Exercices

Quel intérêt ?

Il est tout à fait faisable de faire ses propres fonctions.
Mais pourquoi ?

Quel intérêt ?

Il est tout à fait faisable de faire ses propres fonctions.
Mais pourquoi ?

- ▶ Pour organiser et compartementaliser son code
- ▶ Ne pas réécrire plusieurs fois le même bout de code
- ▶ Isoler des erreurs quand il y en a

La Syntaxe d'une fonction

Une fonction

```
def nom_de_fonction(paramètres_de_fonction):  
    corps_de_fonction # C'est ici qu'on met son code  
    return la_valeur_a_rendre # Ceci est optionnel
```


La Syntaxe d'une fonction

Une fonction

```
def nom_de_fonction(paramètres_de_fonction):  
    corps_de_fonction # C'est ici qu'on met son code  
    return la_valeur_a_rendre # Ceci est optionnel
```

Attention son nom doit être unique !

Quelques fonctions

Une fonction qui ne fait rien

```
def fais_rien():  
    pass
```

Quelques fonctions

Une fonction qui ne fait rien

```
def fais_rien():  
    pass
```

Une fonction qui fait du bruit

```
def dis_quelque_chose():  
    print("coin-coin")
```

Encore des fonctions I

Une fonction qui est toujours d'accord avec nous

```
def daccord():  
    return True  
  
if daccord():  
    print("Super !")  
else:  
    print("Je ne m'attendais pas à ça...")
```

Encore des fonctions II

Demander à un expert en légumes...

```
def legume(couleur):  
    if couleur == 'rouge':  
        return "C'est une tomate"  
    elif couleur == 'vert':  
        return "C'est un poivron"  
    else:  
        return "Je n'ai jamais vu un légume de cette couleur"  
  
print(legume('bleu'))
```

Des fonctions avec plusieurs arguments

Autrement appelés des *arguments positionnels* car **l'ordre est important**

Un menu

```
def menu(vin, entree, plat, dessert):  
    return {'vin':vin, 'entree':entree, 'plat':plat, 'dessert':  
            dessert,}
```

Spécifier des arguments par défaut

Autrement appelés des *arguments positionnels* car **l'ordre est important**

Un menu

```
def menu(vin, entree, plat, dessert="Crème brûlée"):
    return {'vin':vin, 'entree':entree, 'plat':plat, 'dessert':
            dessert,}
```

Revisitons un exercice...

Comment faire la table de multiplication par 7 ?

Rajouter des paramètres

Et si on voulait avoir le tableau de multiplication de n'importe quel chiffre ?

Rajouter des paramètres

Et si on voulait avoir le tableau de multiplication de n'importe quel chiffre ?

Et pour spécifier le début et la fin ?

Montrer tous les tableaux

Faire une boucle pour avoir toutes les tables de 1 à 12
A savoir que le nom du variable qu'on donne comme argument n'est pas important, dans la fonction on aura toujours le nom défini dans la définition de notre fonction

La visibilité des variables

Variable scope en Python

- ▶ **Local** : variable définie dans une fonction.
- ▶ **Enclosing** : variable définie dans une fonction englobante (fonctions imbriquées).
- ▶ **Global** : variable définie au niveau du module.
- ▶ **Built-in** : noms fournis par Python lui-même.

Règle de résolution des noms (LEGB)

- 1 Local
- 2 Enclosing
- 3 Global
- 4 Built-in

Concrètement

Les exemples de visibilité de variables

```
x = "global"

def outer():
    x = "enclosing"

    def inner():
        x = "local"
        print(x)

    inner()
    print(x)

outer()
print(x)
# output : local, puis enclosing puis global
```

Valeur de retour

- ▶ Une fonction renvoie **toujours** une valeur
- ▶ On peut spécifier la valeur à retourner avec return

Valeur de retour

- ▶ Une fonction renvoie **toujours** une valeur
- ▶ On peut spécifier la valeur à retourner avec `return`
- ▶ Si l'on en spécifie pas, la valeur sera `None`

Essayez : `print(print('la valeur renvoyée par cette fonction est None'))`

1 Les Fonctions

2 Faire ses propres fonctions

3 Exercices

Exercice 1 : Fonction de base

Écrivez une fonction appelée `saluer` qui prend le nom d'une personne en paramètre et renvoie un message de salutation.

Exemple :

```
saluer("Alice") # Devrait renvoyer "Bonjour, Alice!"
```

Rappel

```
def nom_de_fonction(paramètres_de_fonction):  
    corps_de_fonction # C'est ici qu'on met son code  
    return la_valeur_a_rendre # Ceci est optionnel
```

Bonus: Rajouter un argument optionnel pour changer *Bonjour*

Exercice 2 : Paramètres multiples

Créez une fonction appelée `calculate_rectangle_area` qui prend deux paramètres (longueur et largeur) et renvoie l'aire d'un rectangle.

Exemple :

```
calculate_rectangle_area(5, 3) # Devrait renvoyer 15
```

Exercice 3 : Logique conditionnelle

Écrivez une fonction appelée `grade_classifier` qui prend une note numérique (0-20) et renvoie la mention correspondante :

- ▶ 16–20 : "Très bien"
- ▶ 14–15.99 : "Bien"
- ▶ 12–13.99 : "Assez bien"
- ▶ 10–11.99 : "Passable"
- ▶ En dessous de 10 : "Insuffisant"

Exemple :

```
grade_classifier(85) # Devrait renvoyer "B"  
grade_classifier(72) # Devrait renvoyer "C"
```

Exercice 4 : Paramètres par défaut

Créez une fonction appelée `trouver_moyenne` qui prend une liste de nombres et un paramètre optionnel `arrondir` (la valeur par défaut est `False`). La fonction doit renvoyer la moyenne des nombres. Si `arrondir` est `True`, arrondissez le résultat à 2 décimales.

Exemple :

```
trouver_moyenne([10, 20, 30]) # Devrait renvoyer 20.0
```

```
trouver_moyenne([10, 15, 15], arrondir=True) # Devrait renvoyer  
13.33
```

Exercice 5 : Composition de fonctions

Écrivez deux fonctions :

- 1 `est_pair(n)` – renvoie `True` si un nombre est premier, `False` sinon
- 2 `compter_paires(start, end)` – utilise `est_pair` pour compter le nombres pairs entre `start` et `end` (inclus)

Exemple :

```
est_pair(7) # Devrait renvoyer False
```

```
est_pair(10) # Devrait renvoyer True
```

Bonus : Pareil mais avec les premiers

eric.jordan@inalco.fr